People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Mohamed Seddik Ben Yahia University - Jijel

Faculty of Sciences and Technology

Department of Computer Science

School Management System

UML Analysis and Design Report

Specialty: Computer Science (Informatics)

Level: 3rd Year License

Academic Session: 2025 - 2026

Prepared by

1. Bouremouz Mouhammed
2. Debieche Amine
3. Bessibes Mouin
4. Chaabani Sidahmed
5. Bouzekria Sohib
6. Meriche Chemseddine

Under the supervision of

Dr. El Hillali Kerkouche

Executive Summary

This project focuses on the conception and development of a collaborative web application for school management. The system is designed for educational support schools, training centers, and language learning institutions, with the objective of simplifying administrative work, improving communication, and supporting academic follow-up.

The application is organized around secure role-based spaces for the main actors of the school: administration, reception, accounting, teachers, students, and parents. Each role has access to the functions that match its responsibilities, including student and parent management, teacher assignments, schedules, attendance, assessments, grades, resources, payments, messages, notifications, and announcements.

The UML documentation models the system through use case diagrams for each role, a class diagram for the backend structure, sequence diagrams for the main runtime interactions, and a navigation diagram that explains how users move between the login page and their dashboards.

Table of Contents

Introduction

This report presents the analysis and design of a School Management System developed according to a web architecture. The application is organized around six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent. These roles represent the main actors involved in school administration, pedagogical monitoring, communication, and financial follow-up.

The documentation focuses on the functional and structural modeling of the project. It includes a project context section, use case diagrams for each role, a class diagram for the backend structure, sequence diagrams for runtime interactions, a navigation diagram, a technology and deployment summary, and a concise conclusion.

Project Context and Benefits

Opening Statement

The School Management System was designed as a practical response to the needs of modern educational institutions. The project aims to connect administration, teachers, parents, students, reception staff, and financial managers inside one coherent web platform. The goal is not only to digitize isolated operations, but also to make school information easier to organize, access, review, and communicate.

System Benefits

The system provides value by reducing repeated manual work, improving the visibility of academic progress, and giving each user a dedicated workspace that matches their role.

Benefit	Contribution to the School Management System
Administrative efficiency	Centralizes student records, parent information, enrollments, schedules, and daily follow-up.
Pedagogical follow-up	Allows teachers, students, and parents to follow assessments, grades, attendance, and resources.
Communication	Improves information exchange through messages, notifications, and administration posts.
Financial clarity	Supports fee tracking, payments, transactions, reminders, and student financial files.
Role-based organization	Separates responsibilities between administration, reception, accounting, teachers, students, and parents.

Initial Data Modeling with Al Abakera Excel Data

At the beginning of the project, an Excel-based data reference related to Al Abakera School was used to understand the real structure of school information. This helped identify the main data entities, such as users, students, parents, teachers, classes, enrollments, fees, payments, attendance, schedules, assessments, grades, and resources.

This early spreadsheet analysis supported the transition from raw operational data to a more structured relational model. The UML class diagram and the backend database managers were then organized around these domains to keep the implementation close to realistic school workflows.

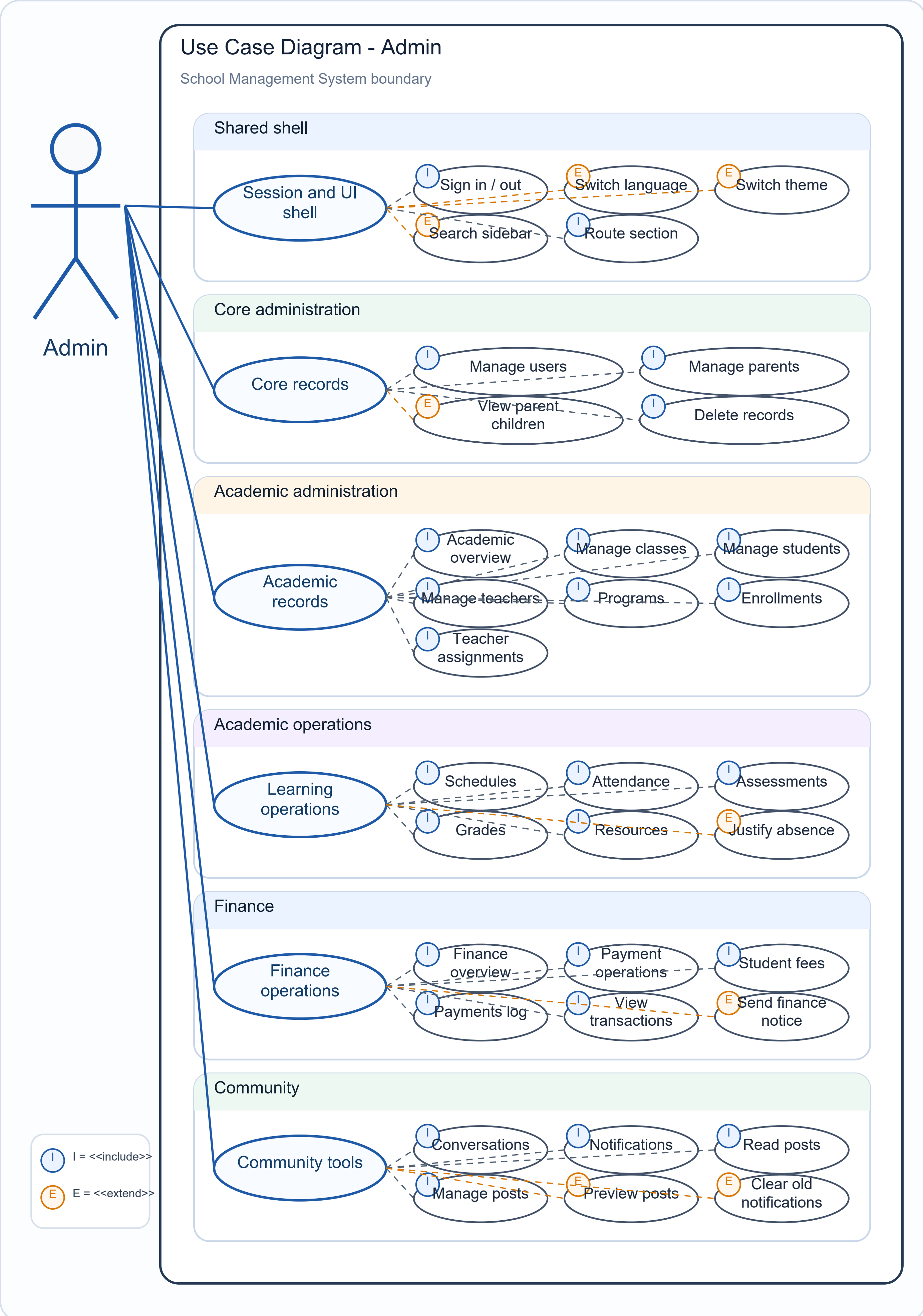
Hosting, Port Forwarding, and Cloud Services

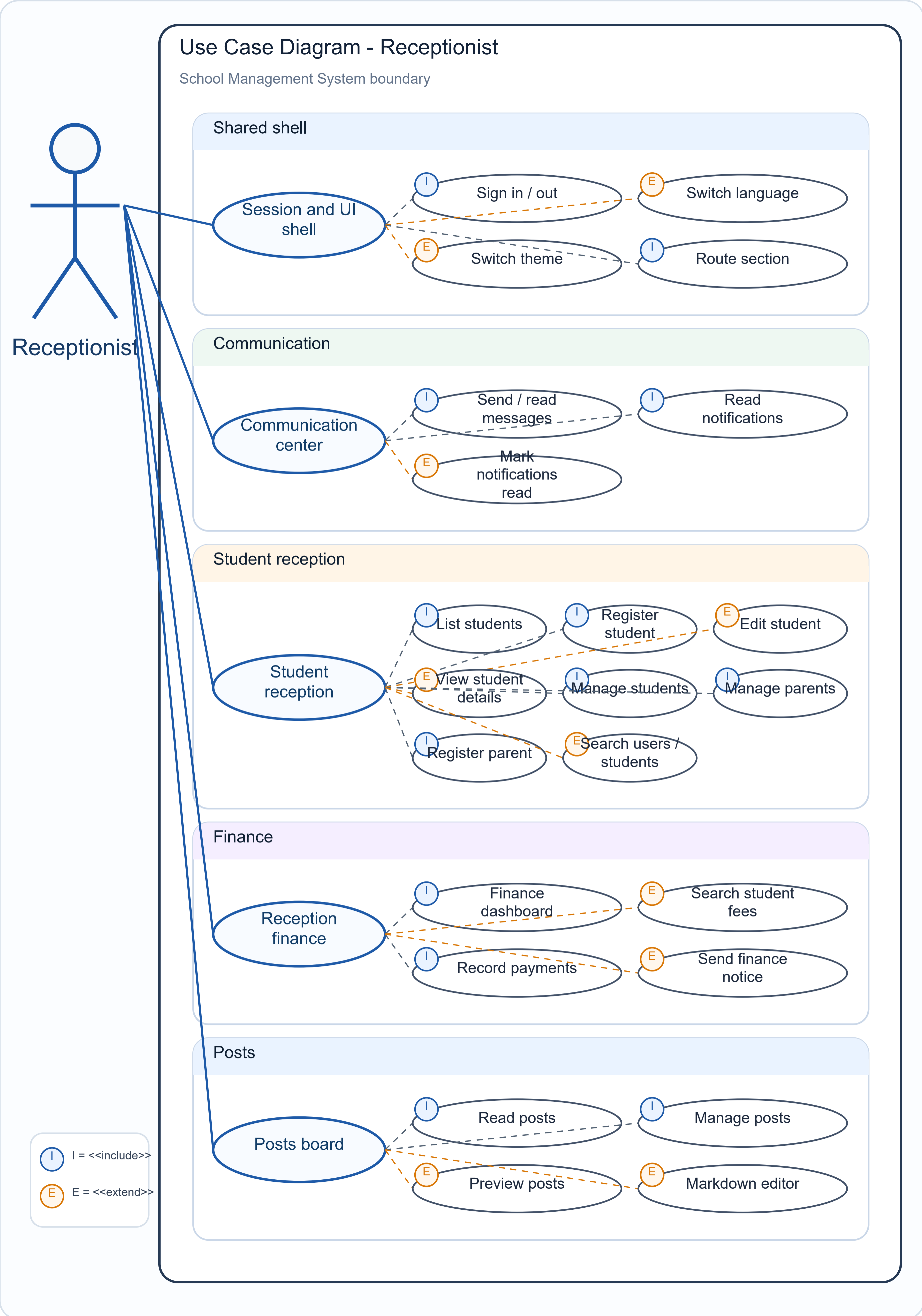
To make the application testable outside the local development environment, hosted access was configured with port forwarding for both the frontend and the backend. This allowed the web interface and the FastAPI service to be reached remotely during testing and demonstration.

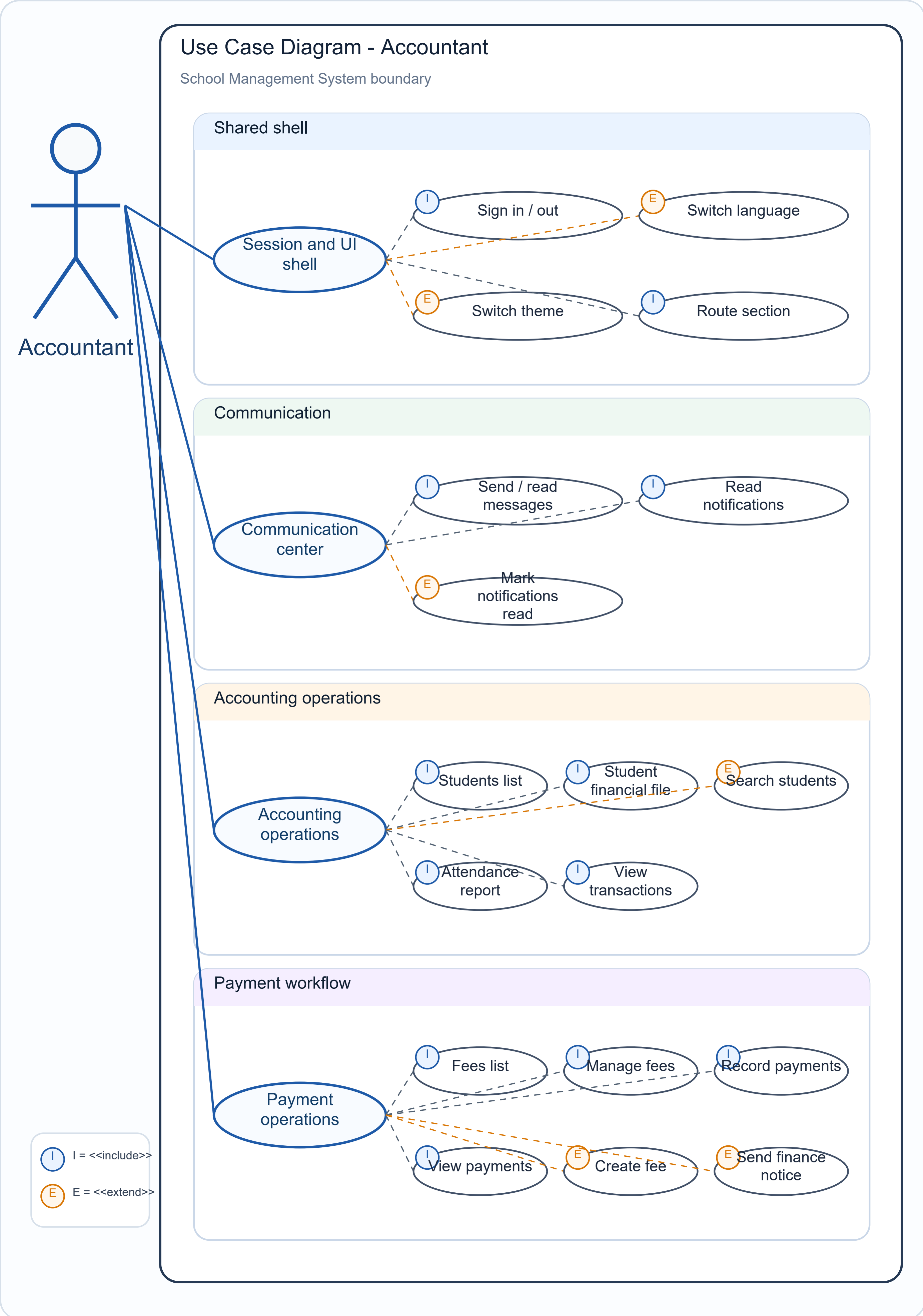
Cloud services were also used to support remote access, availability, and collaboration during the project. This deployment approach reflects the architecture expected from a web-based school management platform: a browser-based frontend communicating with a backend API and a persistent database layer.

Functional Models: Use Case Diagrams

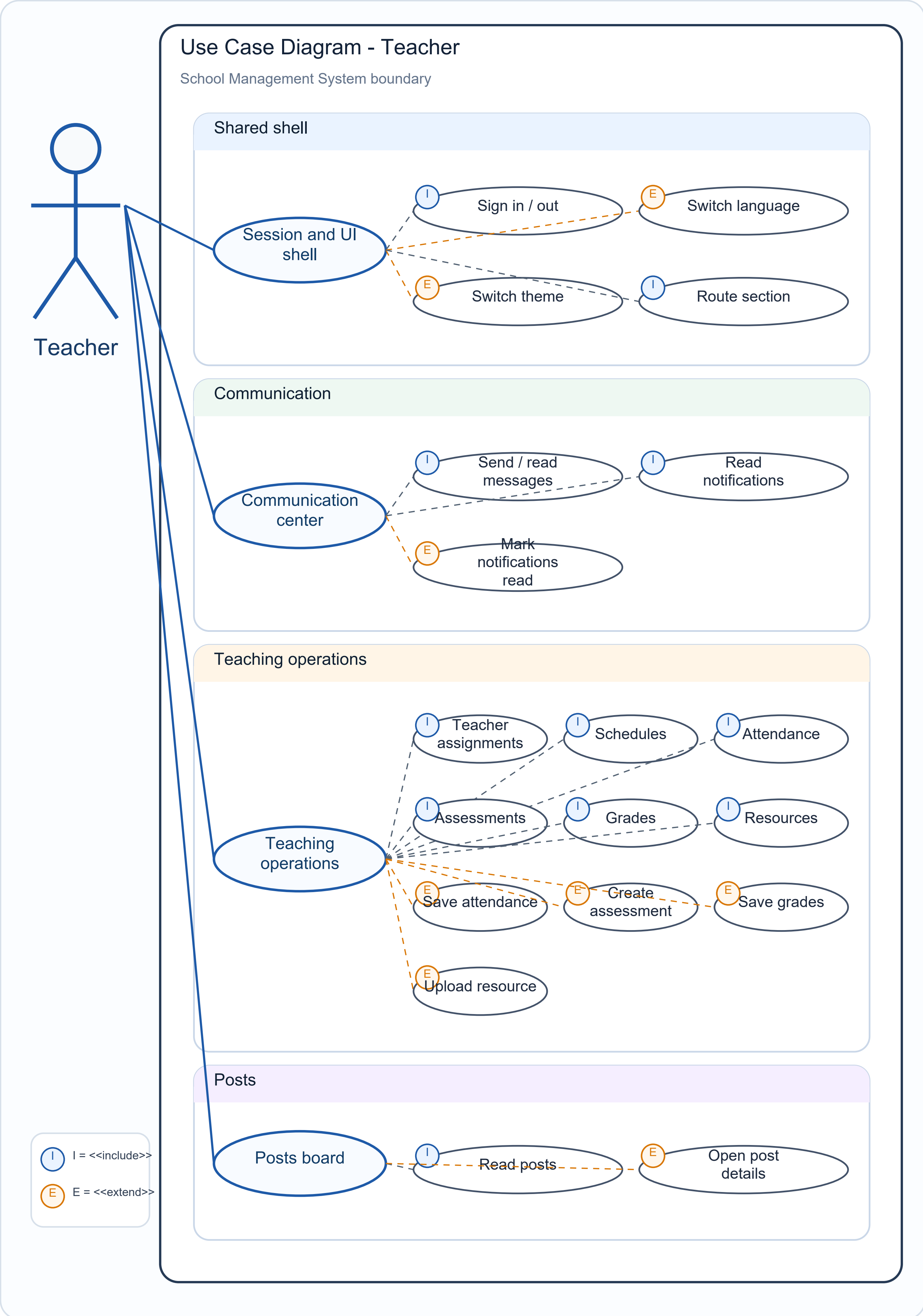
The use case diagrams define the visible services of the School Management System for each role. They show the functional boundary of every dashboard and clarify which operations belong to administration, reception, accounting, teaching, learner self-service, and parent follow-up.

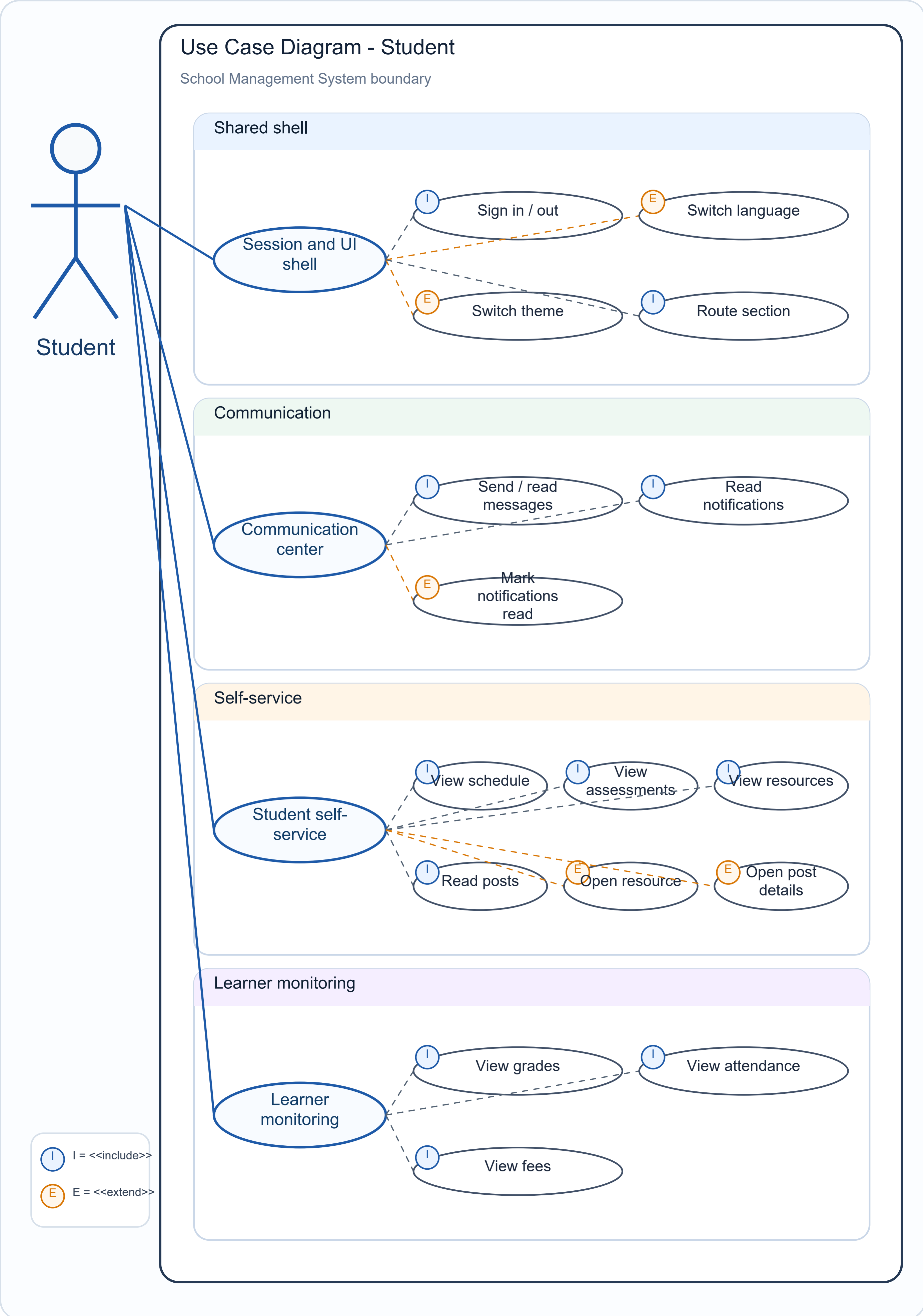

Use Case Diagram - Admin
School Management System boundary
Admin
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Search sidebar
Route section
Core administration
Core records
Manage users
Manage parents
View parent children
Delete records
Academic administration
Academic records
Academic overview
Manage classes
Manage students
Manage teachers
Programs
Enrollments
Teacher assignments
Academic operations
Learning operations
Schedules
Attendance
Assessments
Grades
Resources
Justify absence
Finance
Finance operations
Finance overview
Payment operations
Student fees
Payments log
View transactions
Send finance notice
Community
Community tools
Conversations
Notifications
Read posts
Manage posts
Preview posts
Clear old notifications
I = <<include>>
E = <<extend>>

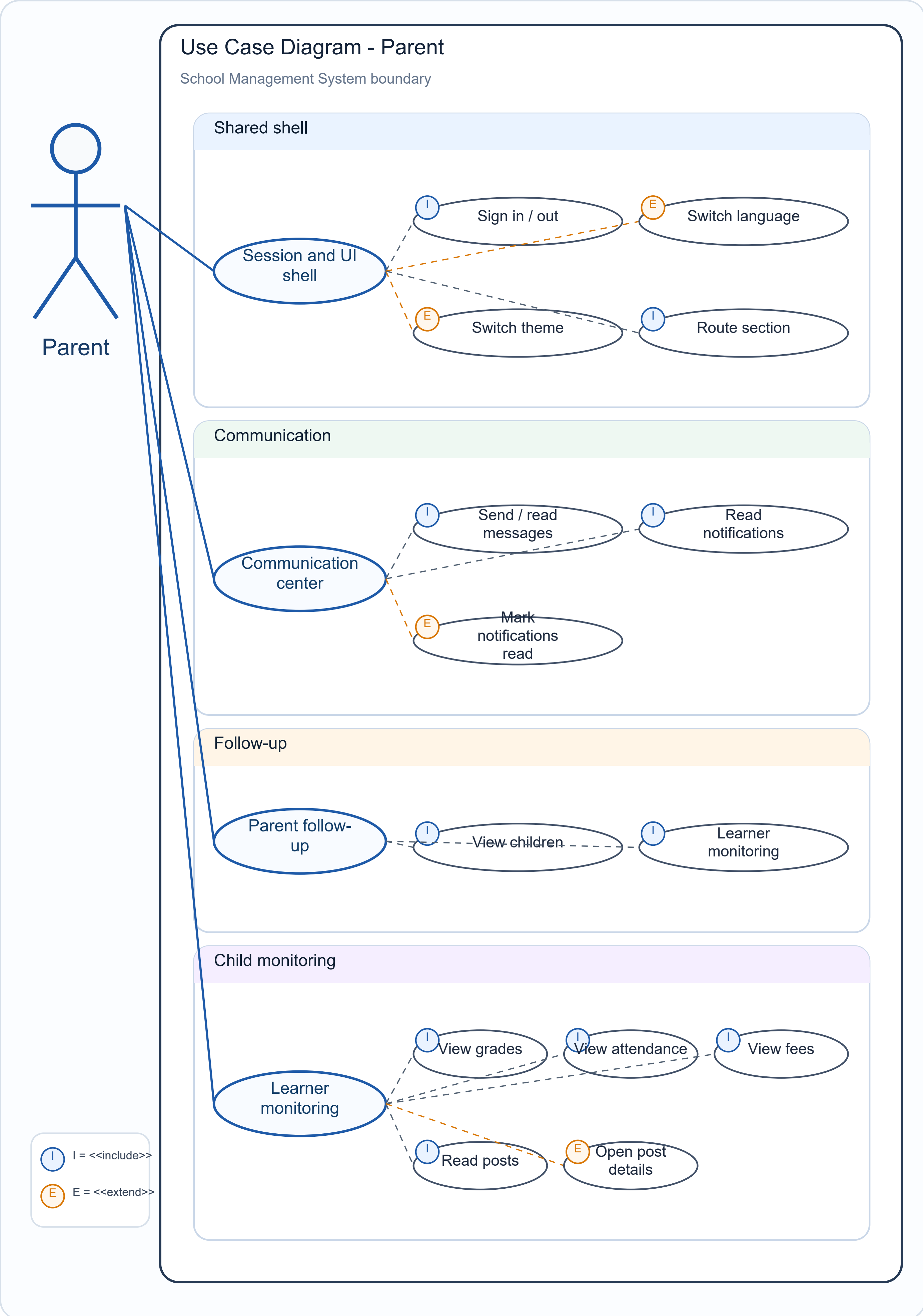

Use Case Diagram - Receptionist
School Management System boundary
Receptionist
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Student reception
Student reception
List students
Register student
Edit student
View student details
Manage students
Manage parents
Register parent
Search users / students
Finance
Reception finance
Finance dashboard
Search student fees
Record payments
Send finance notice
Posts
Posts board
Read posts
Manage posts
Preview posts
Markdown editor
I = <<include>>
E = <<extend>>



Use Case Diagram - Accountant
School Management System boundary
Accountant
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Accounting operations
Accounting operations
Students list
Student financial file
Search students
Attendance report
View transactions
Payment workflow
Payment operations
Fees list
Manage fees
Record payments
View payments
Create fee
Send finance notice
I = <<include>>
E = <<extend>>


Use Case Diagram - Teacher
School Management System boundary
Teacher
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Teaching operations
Teaching operations
Teacher assignments
Schedules
Attendance
Assessments
Grades
Resources
Save attendance
Create assessment
Save grades
Upload resource
Posts
Posts board
Read posts
Open post details
I = <<include>>
E = <<extend>>


Use Case Diagram - Student
School Management System boundary
Student
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Self-service
Student self-service
View schedule
View assessments
View resources
Read posts
Open resource
Open post details
Learner monitoring
Learner monitoring
View grades
View attendance
View fees
I = <<include>>
E = <<extend>>


Use Case Diagram - Parent
School Management System boundary
Parent
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Follow-up
Parent follow-up
View children
Learner monitoring
Child monitoring
Learner monitoring
View grades
View attendance
View fees
Read posts
Open post details
I = <<include>>
E = <<extend>>
I
E

Use Case Scope

Admin: complete system administration across people, academic, finance, posts, and notifications.

Receptionist: front-desk workflows for students, parents, records, finance lookup, posts, and communication.

Accountant: fees, payments, transactions, student financial files, notices, and attendance reports.

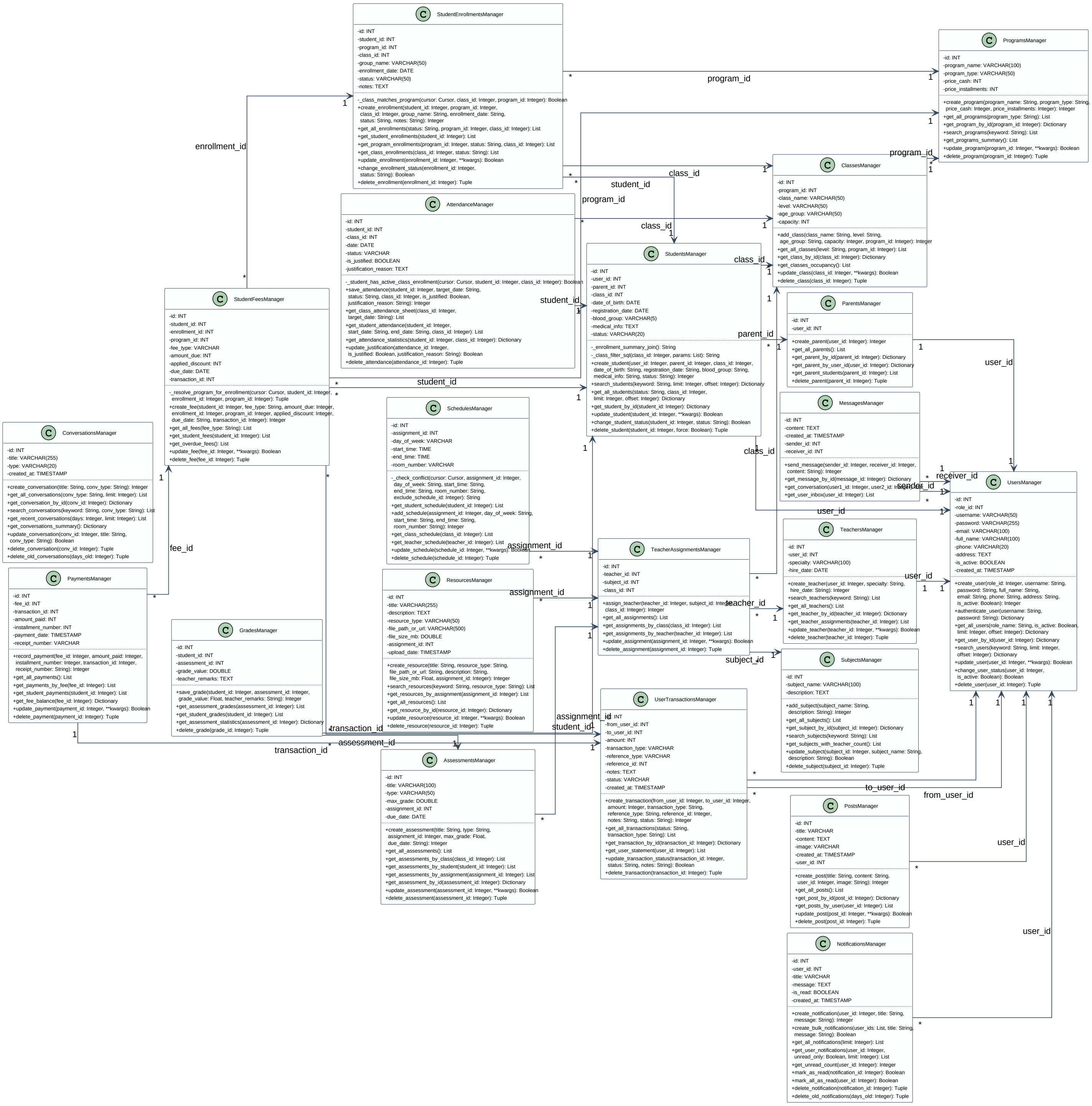
Teacher: assignments, schedules, attendance, assessments, grades, resources, messages, and notifications.

Student: read-only academic self-service: schedule, assessments, grades, attendance, resources, posts, fees, messages, and notifications.

Parent: child monitoring: grades, attendance, fees, posts, messages, and notifications.

Structural Model: Class Diagram

The class diagram presents the backend structure of the School Management System. It groups the database managers and domain entities used to support identity, academic management, learning operations, finance, communication, and publication features.


StudentEnrollmentsManager
ProgramsManager
AttendanceManager
ClassesManager
StudentsManager
StudentFeesManager
ParentsManager
MessagesManager
SchedulesManager
ConversationsManager
UsersManager
TeacherAssignmentsManager
TeachersManager
PaymentsManager
ResourcesManager
GradesManager
SubjectsManager
UserTransactionsManager
AssessmentsManager
PostsManager
NotificationsManager
program_id
enrollment_id
class_id
student_id
parent_id
user_id
receiver_id
sender_id
fee_id
assignment_id
teacher_id
subject_id
transaction_id
assessment_id
to_user_id
from_user_id

Structural Responsibilities

Identity and people: users, parents, students, teachers, authentication, and role ownership.

Academic structure: programs, classes, enrollments, subjects, and teacher assignments.

Learning operations: schedules, attendance, assessments, grades, and educational resources.

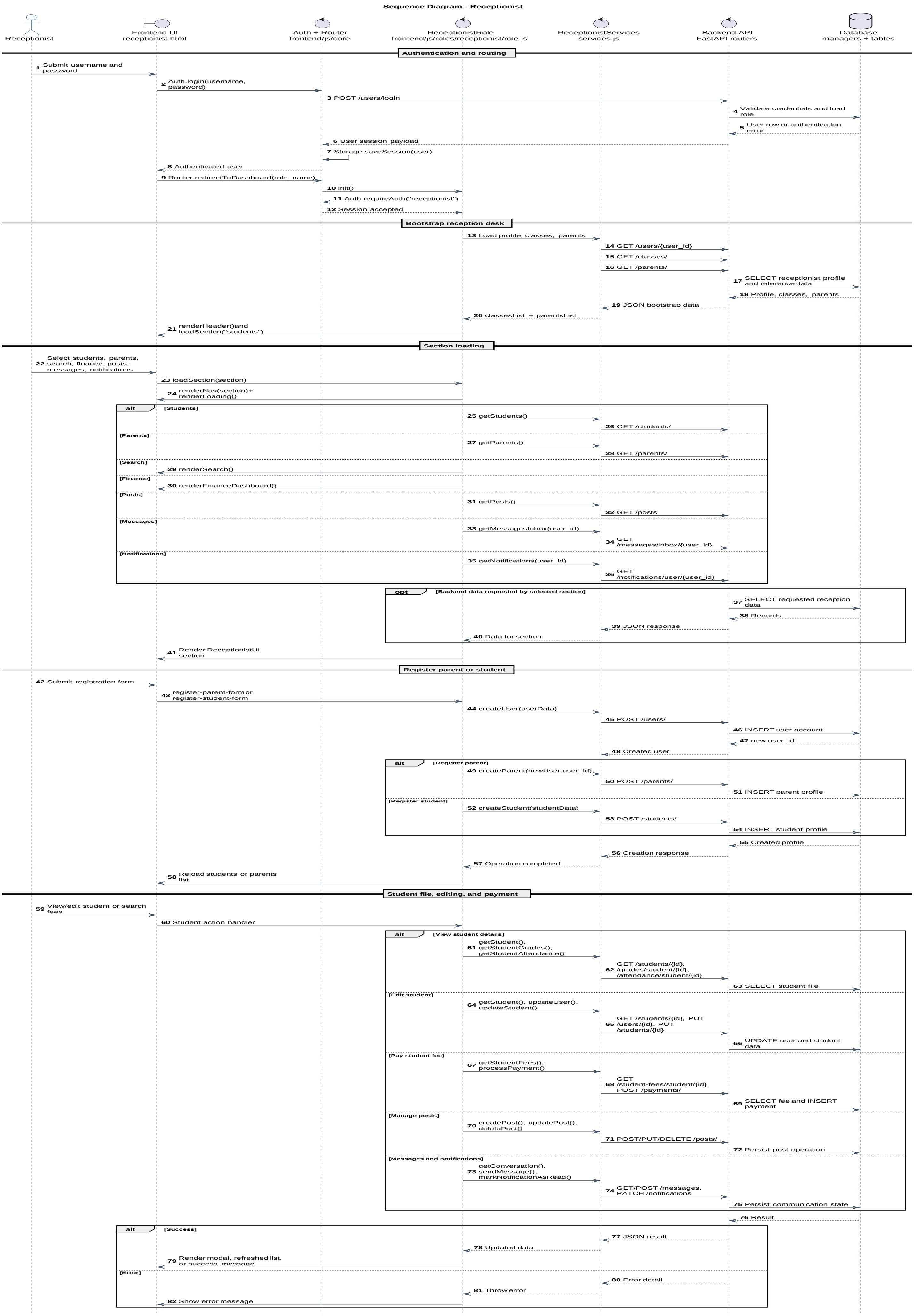
Finance: student fees, payments, and user transactions.

Communication: conversations, messages, notifications, and posts.

Dynamic Models: Sequence Diagrams

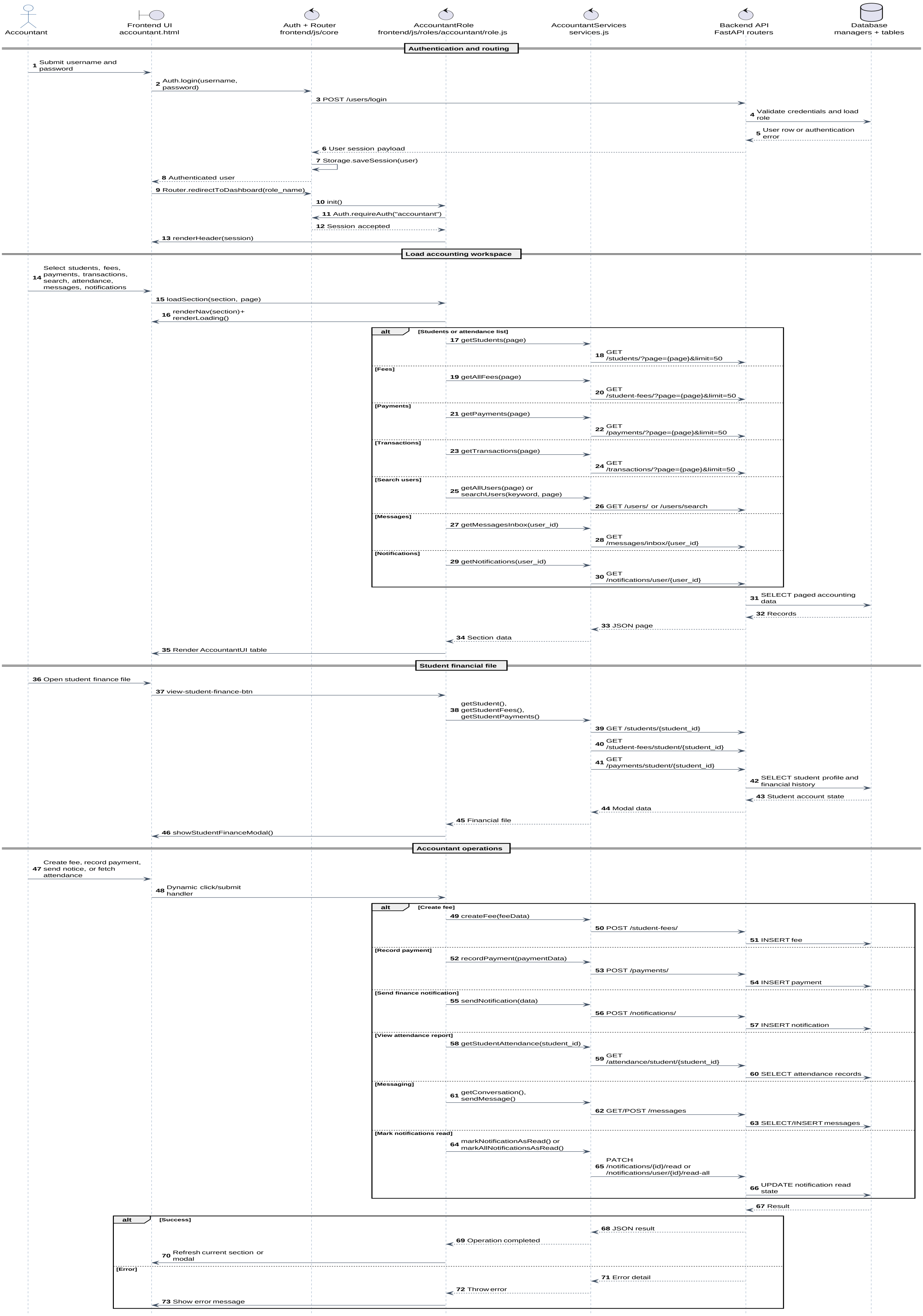
The sequence diagrams describe how each role interacts with the application at runtime. The common pattern is: user action in the frontend, role controller handling, service/API request, FastAPI router processing, database operation, then UI rendering of the result.

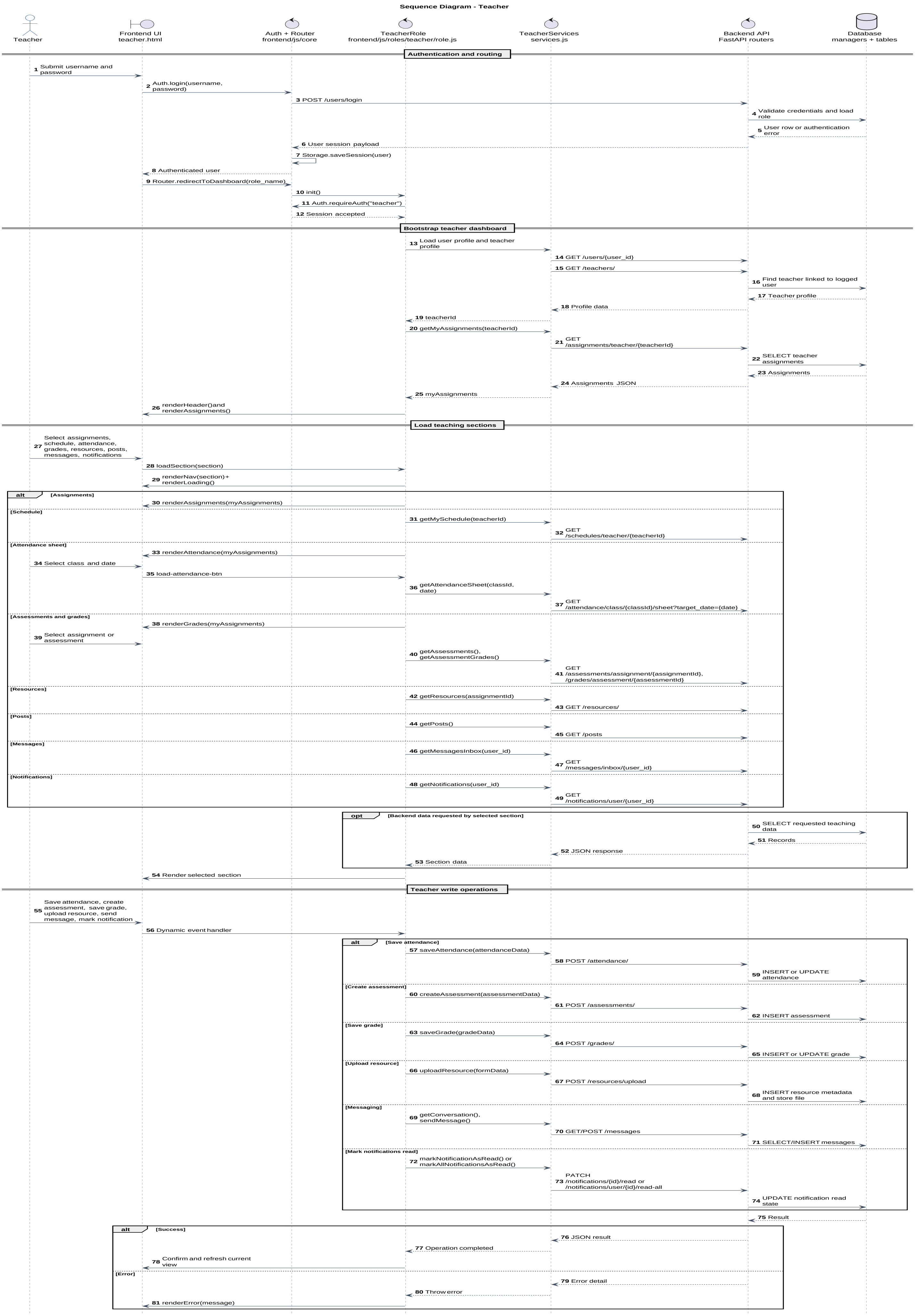

Sequence Diagram - Admin
Admin
Frontend UI
admin.html
Auth + Router
frontend/js/core
AdminRole
frontend/js/roles/admin/role.js
Admin Services/Tabs
services.js + tabs
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("admin")
12 Session accepted
13 renderHeader(session)
14 handleRoute(hash || "academic")
Load selected administration section
15 Select sidebar section
16 loadSection(section)
17 renderNav(section)+ renderLoading()
alt
[Users and people]
18 getUsers(), getParents(), getStudents(), getTeachers()
19 GET /users, /parents, /students, /teachers
[Academic structure]
20 getClasses(), getPrograms(), getEnrollments()
21 GET /classes, /programs, /enrollments
[Learning operations]
22 Load attendance, schedules, assessments, grades, resources
23 GET /attendance, /schedules, /assessments, /grades, /resources
[Finance]
24 getStudentFees(), getPayments(), getTransactions()
25 GET /student-fees, /payments, /transactions
[Community]
26 getConversations(), getNotifications(), getPosts()
27 GET /conversations, /notifications, /posts
28 SELECT requested records
29 Records
30 JSON dataset
31 Dataset for active tab
32 Render AdminUI tab
Admin write operations
33 Submit create/update/delete/status action
34 Call tab handler or deleteItem()
alt
[Create or update core record]
35 POST/PUT users, parents, students, teachers, classes, programs
36 POST/PUT /users, /parents, /students, /teachers, /classes, /programs
37 INSERT or UPDATE core table
[Academic operation]
38 Manage enrollments, schedules, assessments, grades, attendance
39 POST/PUT/PATCH /enrollments, /schedules, /assessments, /grades, /attendance
40 Persist academic operation
[Finance operation]
41 Manage fees, payments, transactions, reminders
42 POST/PUT/PATCH /student-fees, /payments, /transactions, /notifications
43 Persist finance operation
[Community operation]
44 Manage posts, conversations, notifications
45 POST/PUT/PATCH/DELETE /posts, /messages, /notifications
46 Persist community operation
[Delete record]
47 deleteRecord(endpoint, id)
48 DELETE /{endpoint}/{id}
49 DELETE target record
50 Commit result
alt
[Success]
51 Saved/deleted response
52 Operation completed
53 Reload active section and show success
[Validation or authorization error]
54 Error detail
55 Throw error
56 renderError(message)

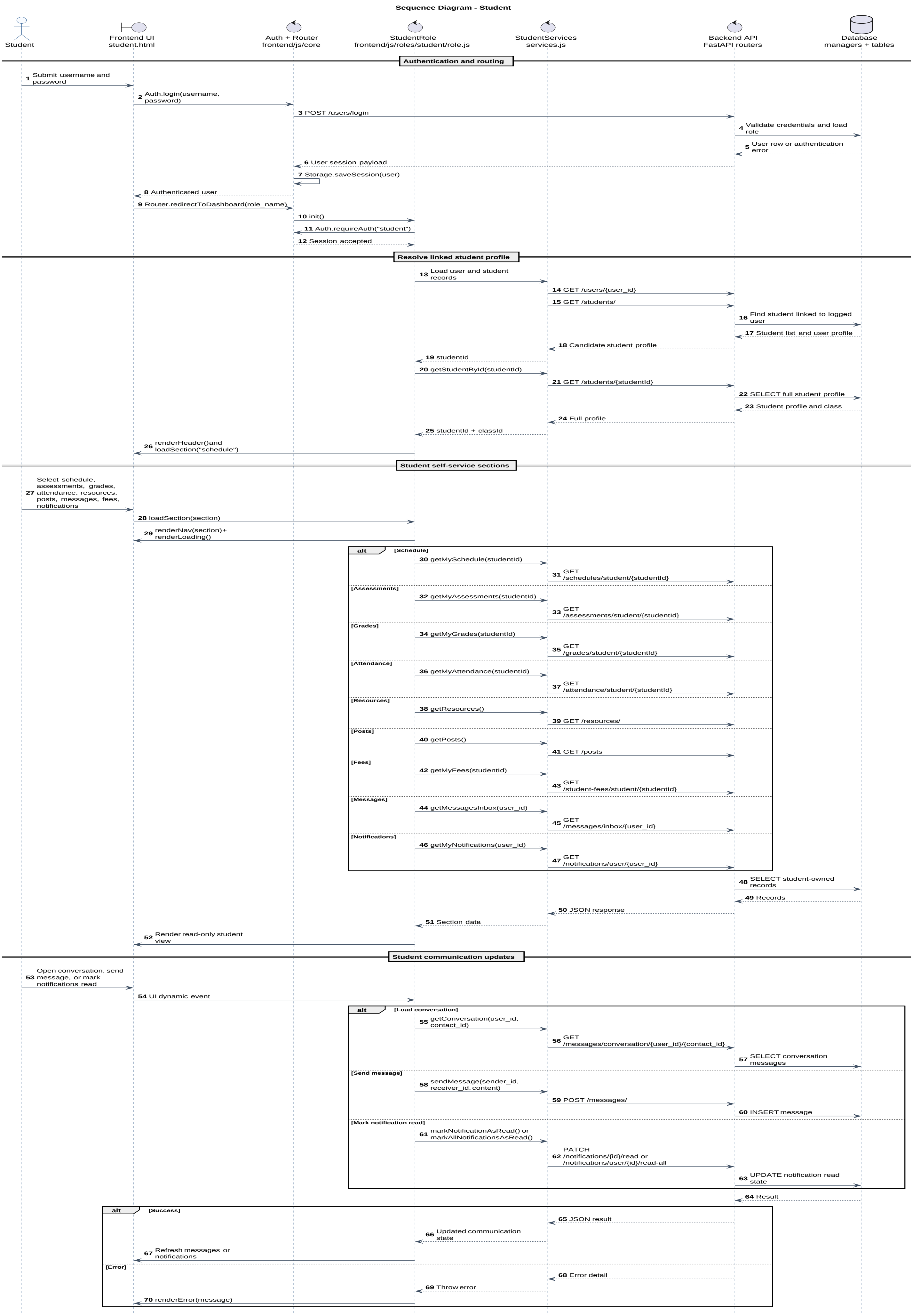


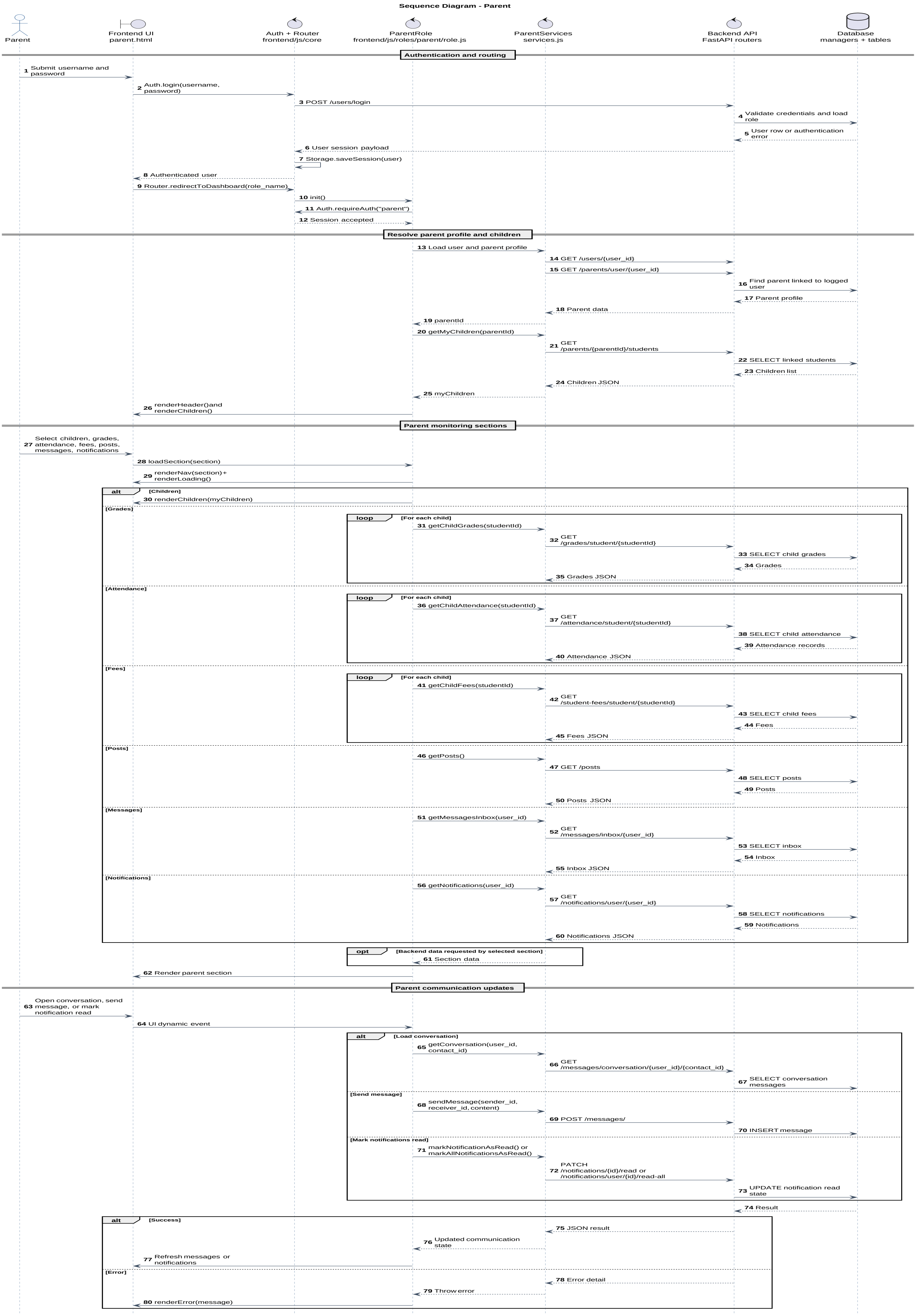
Sequence Diagram - Receptionist
Receptionist
Frontend UI
receptionist.html
Auth + Router
frontend/js/core
ReceptionistRole
frontend/js/roles/receptionist/role.js
ReceptionistServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("receptionist")
12 Session accepted
Bootstrap reception desk
13 Load profile, classes, parents
14 GET /users/{user_id}
15 GET /classes/
16 GET /parents/
17 SELECT receptionist profile and reference data
18 Profile, classes, parents
19 JSON bootstrap data
20 classesList + parentsList
21 renderHeader() and loadSection("students")
Section loading
22 Select students, parents, search, finance, posts, messages, notifications
23 loadSection(section)
24 renderNav(section) + renderLoading()
alt
[Students]
25 getStudents()
26 GET /students/
[Parents]
27 getParents()
28 GET /parents/
[Search]
29 renderSearch()
[Finance]
30 renderFinanceDashboard()
[Posts]
31 getPosts()
32 GET /posts
[Messages]
33 getMessagesInbox(user_id)
34 GET /messages/inbox/{user_id}
[Notifications]
35 getNotifications(user_id)
36 GET /notifications/user/{user_id}
opt
[Backend data requested by selected section]
37 SELECT requested reception data
38 Records
39 JSON response
40 Data for section
41 Render ReceptionistUI section
Register parent or student
42 Submit registration form
43 register-parent-form or register-student-form
44 createUser(userData)
45 POST /users/
46 INSERT user account
47 new user_id
48 Created user
alt
[Register parent]
49 createParent(newUser.user_id)
50 POST /parents/
51 INSERT parent profile
[Register student]
52 createStudent(studentData)
53 POST /students/
54 INSERT student profile
55 Created profile
56 Creation response
57 Operation completed
58 Reload students or parents list
Student file, editing, and payment
59 View/edit student or search fees
60 Student action handler
alt
[View student details]
61 getStudent(), getStudentGrades(), getStudentAttendance()
62 GET /students/{id}, /grades/student/{id}, /attendance/student/{id}
63 SELECT student file
[Edit student]
64 getStudent(), updateUser(), updateStudent()
65 GET /students/{id}, PUT /users/{id}, PUT /students/{id}
66 UPDATE user and student data
[Pay student fee]
67 getStudentFees(), processPayment()
68 GET /student-fees/student/{id}, POST /payments/
69 SELECT fee and INSERT payment
[Manage posts]
70 createPost(), updatePost(), deletePost()
71 POST/PUT/DELETE /posts/
72 Persist post operation
[Messages and notifications]
73 getConversation(), sendMessage(), markNotificationAsRead()
74 GET/POST /messages, PATCH /notifications
75 Persist communication state
76 Result
alt
[Success]
77 JSON result
78 Updated data
79 Render modal, refreshed list, or success message
[Error]
80 Error detail
81 Throw error
82 Show error message

Sequence Diagram - Accountant




Sequence Diagram - Teacher
Teacher
Frontend UI
teacher.html
Auth + Router
frontend/js/core
TeacherRole
frontend/js/roles/teacher/role.js
TeacherServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("teacher")
12 Session accepted
Bootstrap teacher dashboard
13 Load user profile and teacher profile
14 GET /users/{user_id}
15 GET /teachers/
16 Find teacher linked to logged user
17 Teacher profile
18 Profile data
19 teacherId
20 getMyAssignments(teacherId)
21 GET /assignments/teacher/{teacherId}
22 SELECT teacher assignments
23 Assignments
24 Assignments JSON
25 myAssignments
26 renderHeader()and renderAssignments()
Load teaching sections
27 Select assignments, schedule, attendance, grades, resources, posts, messages, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt
[Assignments]
30 renderAssignments(myAssignments)
[Schedule]
31 getMySchedule(teacherId)
32 GET /schedules/teacher/{teacherId}
[Attendance sheet]
33 renderAttendance(myAssignments)
34 Select class and date
35 load-attendance-btn
36 getAttendanceSheet(classId, date)
37 GET /attendance/class/{classId}/sheet?target_date={date}
[Assessments and grades]
38 renderGrades(myAssignments)
39 Select assignment or assessment
40 getAssessments(), getAssessmentGrades()
41 GET /assessments/assignment/{assignmentId}, /grades/assessment/{assessmentId}
[Resources]
42 getResources(assignmentId)
43 GET /resources/
[Posts]
44 getPosts()
45 GET /posts
[Messages]
46 getMessagesInbox(user_id)
47 GET /messages/inbox/{user_id}
[Notifications]
48 getNotifications(user_id)
49 GET /notifications/user/{user_id}
opt
[Backend data requested by selected section]
50 SELECT requested teaching data
51 Records
52 JSON response
53 Section data
54 Render selected section
Teacher write operations
55 Save attendance, create assessment, save grade, upload resource, send message, mark notification
56 Dynamic event handler
alt
[Save attendance]
57 saveAttendance(attendanceData)
58 POST /attendance/
59 INSERT or UPDATE attendance
[Create assessment]
60 createAssessment(assessmentData)
61 POST /assessments/
62 INSERT assessment
[Save grade]
63 saveGrade(gradeData)
64 POST /grades/
65 INSERT or UPDATE grade
[Upload resource]
66 uploadResource(formData)
67 POST /resources/upload
68 INSERT resource metadata and store file
[Messaging]
69 getConversation(), sendMessage()
70 GET/POST /messages
71 SELECT/INSERT messages
[Mark notifications read]
72 markNotificationAsRead() or markAllNotificationsAsRead()
73 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
74 UPDATE notification read state
75 Result
alt
[Success]
76 JSON result
77 Operation completed
78 Confirm and refresh current view
[Error]
79 Error detail
80 Throw error
81 renderError(message)


Sequence Diagram - Student
Student
Frontend UI
student.html
Auth + Router
frontend/js/core
StudentRole
frontend/js/roles/student/role.js
StudentServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("student")
12 Session accepted
Resolve linked student profile
13 Load user and student records
14 GET /users/{user_id}
15 GET /students/
16 Find student linked to logged user
17 Student list and user profile
18 Candidate student profile
19 studentId
20 getStudentById(studentId)
21 GET /students/{studentId}
22 SELECT full student profile
23 Student profile and class
24 Full profile
25 studentId + classId
26 renderHeader()and loadSection("schedule")
Student self-service sections
27 Select schedule, assessments, grades, attendance, resources, posts, messages, fees, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt
[Schedule]
30 getMySchedule(studentId)
31 GET /schedules/student/{studentId}
[Assessments]
32 getMyAssessments(studentId)
33 GET /assessments/student/{studentId}
[Grades]
34 getMyGrades(studentId)
35 GET /grades/student/{studentId}
[Attendance]
36 getMyAttendance(studentId)
37 GET /attendance/student/{studentId}
[Resources]
38 getResources()
39 GET /resources/
[Posts]
40 getPosts()
41 GET /posts
[Fees]
42 getMyFees(studentId)
43 GET /student-fees/student/{studentId}
[Messages]
44 getMessagesInbox(user_id)
45 GET /messages/inbox/{user_id}
[Notifications]
46 getMyNotifications(user_id)
47 GET /notifications/user/{user_id}
48 SELECT student-owned records
49 Records
50 JSON response
51 Section data
52 Render read-only student view
Student communication updates
53 Open conversation, send message, or mark notifications read
54 UI dynamic event
alt
[Load conversation]
55 getConversation(user_id, contact_id)
56 GET /messages/conversation/{user_id}/{contact_id}
57 SELECT conversation messages
[Send message]
58 sendMessage(sender_id, receiver_id, content)
59 POST /messages/
60 INSERT message
[Mark notification read]
61 markNotificationAsRead() or markAllNotificationsAsRead()
62 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
63 UPDATE notification read state
64 Result
alt
[Success]
65 JSON result
66 Updated communication state
67 Refresh messages or notifications
[Error]
68 Error detail
69 Throw error
70 renderError(message)



Sequence Diagram - Parent
Parent
Frontend UI parent.html
Auth + Router frontend/js/core
ParentRole frontend/js/roles/parent/role.js
ParentServices services.js
Backend API FastAPI routers
Database managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("parent")
12 Session accepted
Resolve parent profile and children
13 Load user and parent profile
14 GET /users/{user_id}
15 GET /parents/user/{user_id}
16 Find parent linked to logged user
17 Parent profile
18 Parent data
19 parentId
20 getMyChildren(parentId)
21 GET /parents/{parentId}/students
22 SELECT linked students
23 Children list
24 Children JSON
25 myChildren
26 renderHeader()and renderChildren()
Parent monitoring sections
27 Select children, grades, attendance, fees, posts, messages, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt [Children]
30 renderChildren(myChildren)
[Grades]
loop [For each child]
31 getChildGrades(studentId)
32 GET /grades/student/{studentId}
33 SELECT child grades
34 Grades
35 Grades JSON
[Attendance]
loop [For each child]
36 getChildAttendance(studentId)
37 GET /attendance/student/{studentId}
38 SELECT child attendance
39 Attendance records
40 Attendance JSON
[Fees]
loop [For each child]
41 getChildFees(studentId)
42 GET /student-fees/student/{studentId}
43 SELECT child fees
44 Fees
45 Fees JSON
[Posts]
46 getPosts()
47 GET /posts
48 SELECT posts
49 Posts
50 Posts JSON
[Messages]
51 getMessagesInbox(user_id)
52 GET /messages/inbox/{user_id}
53 SELECT inbox
54 Inbox
55 Inbox JSON
[Notifications]
56 getNotifications(user_id)
57 GET /notifications/user/{user_id}
58 SELECT notifications
59 Notifications
60 Notifications JSON
opt [Backend data requested by selected section]
61 Section data
62 Render parent section
Parent communication updates
63 Open conversation, send message, or mark notification read
64 UI dynamic event
alt [Load conversation]
65 getConversation(user_id, contact_id)
66 GET /messages/conversation/{user_id}/{contact_id}
67 SELECT conversation messages
[Send message]
68 sendMessage(sender_id, receiver_id, content)
69 POST /messages/
70 INSERT message
[Mark notifications read]
71 markNotificationAsRead() or markAllNotificationsAsRead()
72 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
73 UPDATE notification read state
74 Result
alt [Success]
75 JSON result
76 Updated communication state
77 Refresh messages or notifications
[Error]
78 Error detail
79 Throw error
80 renderError(message)

Sequence Flow Summary

Admin: authentication, route selection, data loading, and full create/update/delete administration.

Receptionist: dashboard bootstrap, student and parent registration, student records, payments, posts, messaging, and notifications.

Accountant: finance workspace loading, student financial files, fee creation, payment recording, notices, attendance reports, messaging, and notification updates.

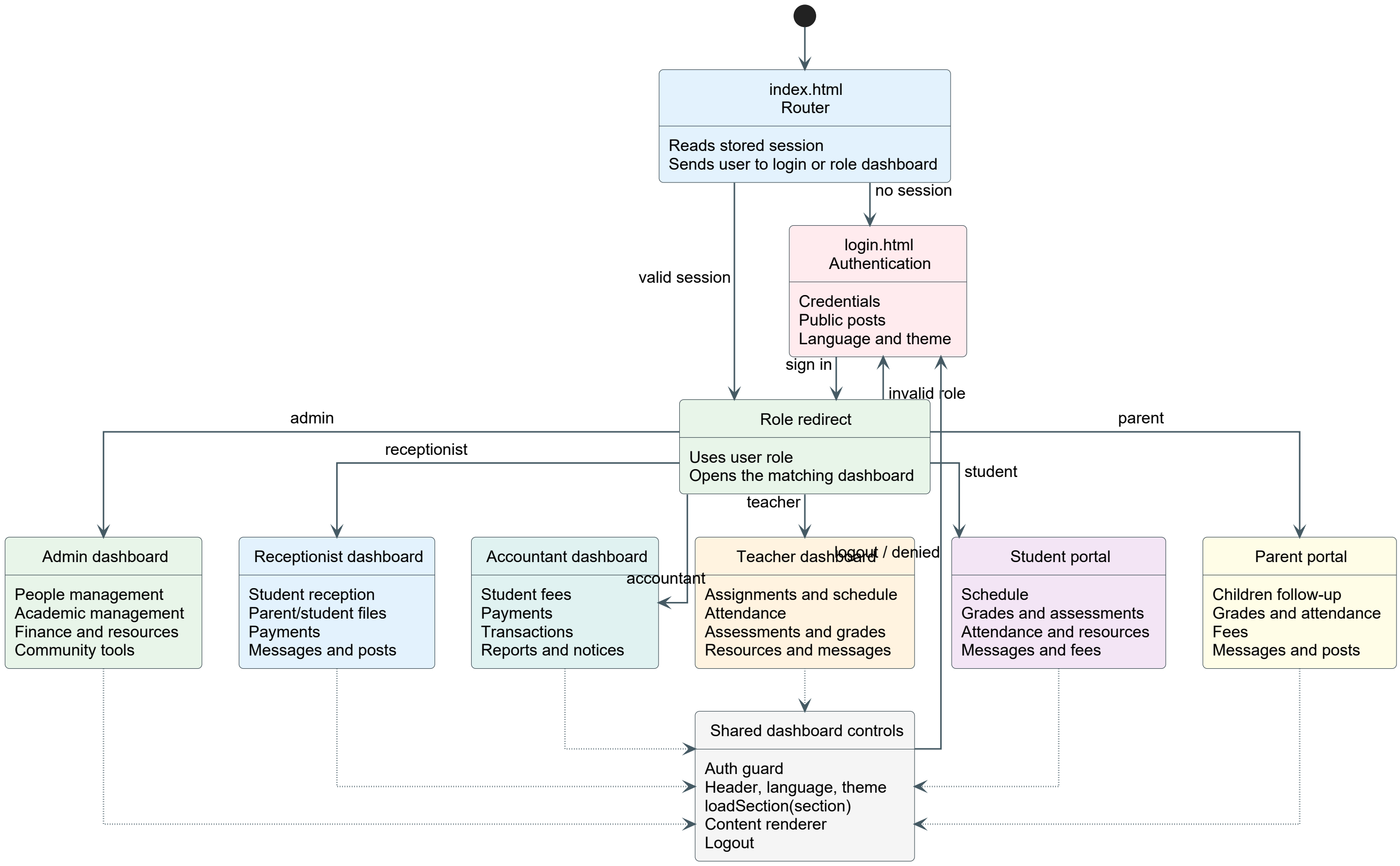
Teacher: teacher profile resolution, assignments, schedules, attendance, assessments, grades, resources, messaging, and notifications.

Student: linked profile resolution, academic self-service views, messages, and notifications.

Parent: parent profile resolution, linked children aggregation, child monitoring, posts, messages, and notifications.

User Interface: Navigation Diagram

The navigation diagram documents how users move from the entry point and authentication page into their role-specific dashboards. It also shows the shared frontend controls and the major section families exposed by each role.


index.html
Router
Reads stored session
Sends user to login or role dashboard
no session
valid session
login.html
Authentication
Credentials
Public posts
Language and theme
sign in
invalid role
Role redirect
Uses user role
Opens the matching dashboard
admin
receptionist
parent
student
teacher
accountant
logout / denied
Admin dashboard
People management
Academic management
Finance and resources
Community tools
Receptionist dashboard
Student reception
Parent/student files
Payments
Messages and posts
Accountant dashboard
Student fees
Payments
Transactions
Reports and notices
Teacher dashboard
Assignments and schedule
Attendance
Assessments and grades
Resources and messages
Student portal
Schedule
Grades and assessments
Attendance and resources
Messages and fees
Parent portal
Children follow-up
Grades and attendance
Fees
Messages and posts
Shared dashboard controls
Auth guard
Header, language, theme
loadSection(section)
Content renderer
Logout

Navigation Flow

Entry: users start from the index/login flow, where authentication determines the target dashboard.

Dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent each have a dedicated page and role controller.

Sections: navigation buttons call the active role controller, which loads the selected section and renders the corresponding UI.

Shared controls: authentication checks, language switching, theme preferences, logout, messaging, and notifications are reused across dashboards.

Technology Stack, Deployment, and Repository

This section lists the technologies directly used by the School Management System, its UML report, and its hosted demonstration setup.

Frontend Layer

The frontend is a static web application built with **HTML5**, **CSS3**, and **vanilla JavaScript**. It uses shared core modules for routing, authentication, local storage, API calls, preferences, and internationalization. Each role has its own page, controller, service layer, and UI renderer.

Backend API Layer

The backend is built with **Python** and **FastAPI**. The API is organized into routers for users, parents, students, teachers, classes, enrollments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, notifications, messages, posts, and programs. **Uvicorn** is used as the ASGI server and **Pydantic** supports request/response validation.

Database and Persistence

The persistence layer uses a relational database accessed through **mysql-connector-python** and **SQLAlchemy** utilities. Database managers separate identity, academic, learning, finance, and communication responsibilities.

Documentation and UML Report

UML diagrams are exported as **SVG** assets and embedded into the PDF report with **ReportLab** and **svglib**. The report covers use case diagrams, the class diagram, sequence diagrams, and the navigation diagram.

Hosting and Cloud Access

The application was prepared for hosted access by configuring port forwarding for both the frontend and backend services. This made it possible to demonstrate and test the web interface and the FastAPI API from outside the local development environment. Cloud services were used to support remote availability, collaboration, and practical validation of the web architecture.

Artificial Intelligence Assistance

AI-assisted tools were used during the project to support analysis, code review, UML/report refinement, and documentation improvement. The final implementation and modeling decisions remain part of the team project work.

Project Repository

The project source code is available on GitHub:
https://github.com/MohammedBoure/SMS-FE-BE

Conclusion

Project Summary

The UML documentation provides a complete view of the School Management System and its main responsibilities. The use case diagrams define role responsibilities, the class diagram explains backend structure, the sequence diagrams show runtime frontend-to-backend collaboration, and the navigation diagram clarifies how users move through the interface.

Design Value

Together, the diagrams make the system easier to understand, review, and extend. They separate user responsibilities clearly, expose the core backend domains, and document the flow of data between the browser, API layer, and database. The project also demonstrates a practical school workflow: data structure was first studied through the Al Abakera Excel reference, then transformed into UML models, backend APIs, role-based frontend dashboards, and a hosted web setup.

Operational Benefits

The system improves daily school management by centralizing academic, administrative, financial, and communication tasks. It gives teachers tools for resources, attendance, assessments, and grades; gives students and parents clearer follow-up; and gives administration, reception, and accounting staff a more organized workspace for records, payments, schedules, notifications, and reports.

Repository and Tooling Note

The project repository is available at **https://github.com/MohammedBoure/SMS-FE-BE**. AI-assisted tools were used to support development analysis, documentation refinement, and UML/report quality improvements.